

CHAPTER 5: CLIENT LOGIC & INTEGRATION

5.1 Introduction to Client Logic and Integration

5.2 Objectives of Client Logic and Integration

5.3 Client-Side Architecture

5.4 Input Validation Implementation

5.5 Financial Data Processing

5.6 Claude AI Integration

5.7 Prompt Engineering Strategies

5.8 Google Sheets Integration

5.9 API Communication Protocol

5.10 Dynamic Dashboard Updates

5.11 Security Measures

5.12 Error Handling and Recovery

5.13 Advantages of Client Logic and Integration

5.14 Performance Optimization

5.15 Future Enhancements

5.16 Chapter Summary

CHAPTER 5: CLIENT LOGIC & INTEGRATION

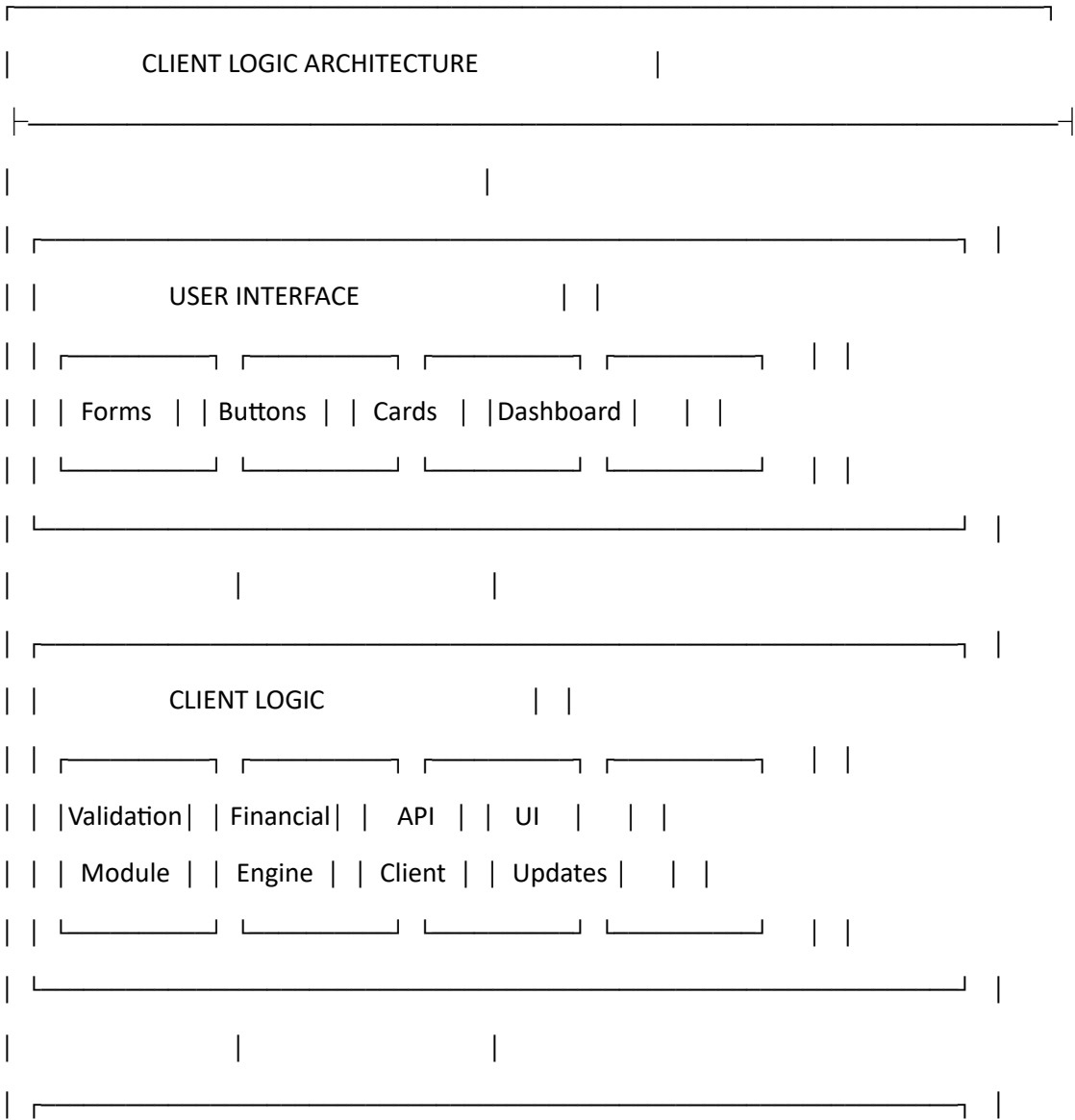
5.1 Introduction to Client Logic and Integration

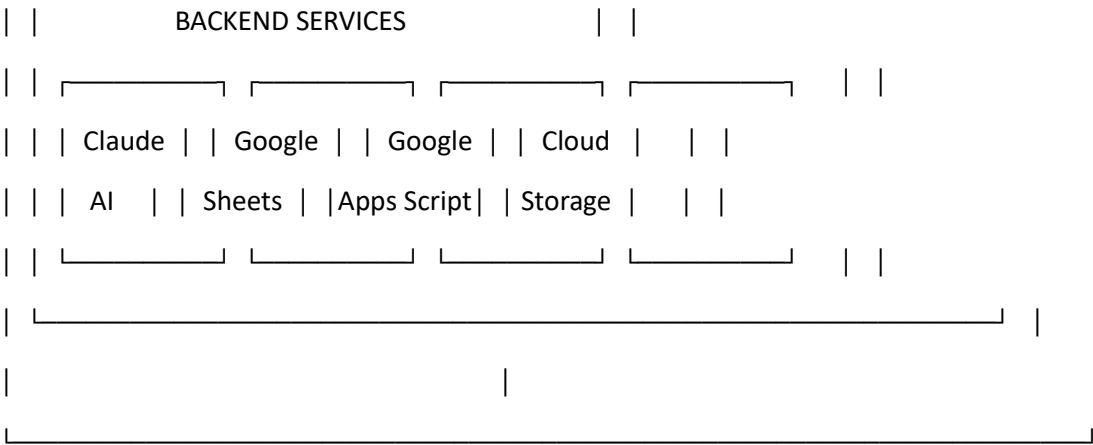
Client Logic and Integration is one of the most important phases in the development of the FinWise AI platform. After designing the user interface and implementing the core financial modules, the next step is to connect these modules with Artificial Intelligence services, cloud databases, validation mechanisms, and external APIs.

Without proper integration, individual components would operate independently, making the application incapable of providing a seamless user experience. The client logic layer acts as the bridge between the user interface and backend services, ensuring smooth communication and data flow throughout the application.

Integration Layers

text





5.2 Objectives of Client Logic and Integration

Primary Objectives

Objective	Description
Data Validation	Ensure all user inputs are valid before processing
Financial Processing	Perform accurate financial calculations
AI Integration	Connect with Claude AI for intelligent recommendations
Cloud Storage	Integrate with Google Sheets for data persistence
API Communication	Handle all external API interactions
Dynamic Updates	Update the UI in real-time
Error Handling	Manage errors gracefully
Security	Implement security measures at client level

Integration Goals

Goal	Metric	Target
Reliability	Error-free processing	> 99%
Speed	Response time	< 2 seconds
Data Accuracy	Calculation precision	100%
User Satisfaction	User feedback	> 90%

5.3 Client-Side Architecture

JavaScript-Based Architecture

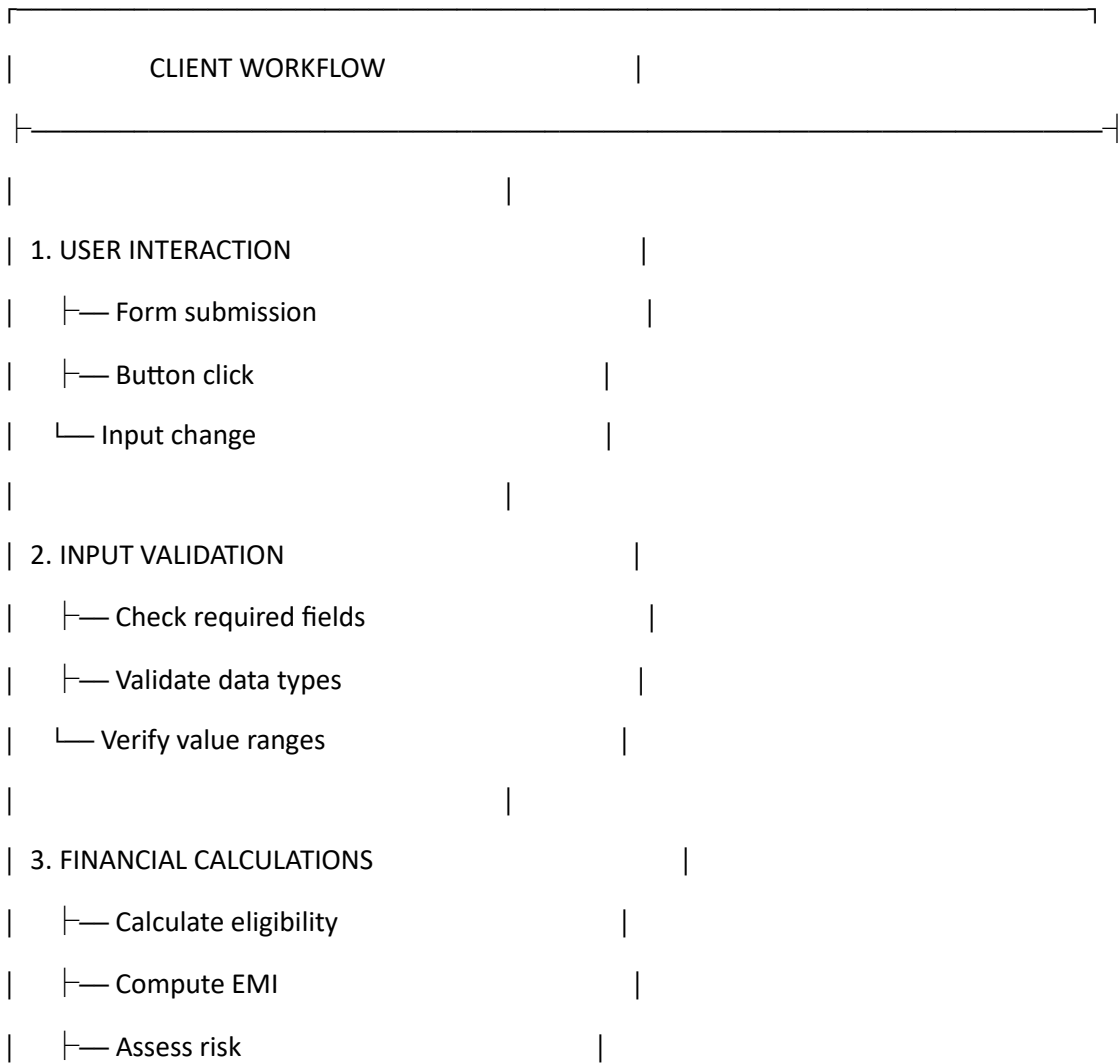
The client logic is implemented entirely using JavaScript, controlling application behavior after user interaction.

Core Components:

Component	File	Responsibility
Main Controller	main.js	Application initialization, routing
Validation Module	validation.js	Input validation and error handling
Financial Engine	financial.js	Financial calculations and assessments
API Client	api.js	Communication with external services
UI Manager	ui.js	DOM manipulation and updates
Dashboard Manager	dashboard.js	Dashboard updates and visualization

Workflow

text





5.4 Input Validation Implementation

Validation Process

javascript

```
class FormValidator {  
  constructor() {  
    this.validationRules = this.getValidationRules();  
  }  
  
  validateField(field, value) {  
    const rules = this.validationRules[field];  
    const errors = [];
```

```

    for (const rule of rules) {
      const result = rule.check(value);
      if (!result.valid) {
        errors.push(result.message);
      }
    }

    return errors;
  }

  validateForm(data) {
    const errors = {};
    for (const field in data) {
      const fieldErrors = this.validateField(field, data[field]);
      if (fieldErrors.length > 0) {
        errors[field] = fieldErrors;
      }
    }
    return errors;
  }
}

```

Validation Rules

Field	Rule	Error Message
income	Required, numeric, > 0	"Please enter your monthly income"
age	Required, numeric, 18-70	"Age must be between 18 and 70"
creditScore	Required, numeric, 300-900	"Credit score must be between 300 and 900"
loanAmount	Required, numeric, 10000-10000000	"Loan amount must be between ₹10,000 and ₹10,000,000"
tenure	Required, numeric, 6-360	"Tenure must be between 6 and 360 months"

Real-time Validation

javascript

// Real-time validation on input

```
document.getElementById('income').addEventListener('input', function(e) {  
    const value = e.target.value;  
    const errors = validator.validateField('income', value);  
  
    const errorElement = document.getElementById('income-error');  
    if (errors.length > 0) {  
        errorElement.textContent = errors[0];  
        errorElement.style.display = 'block';  
        e.target.classList.add('error');  
    } else {  
        errorElement.style.display = 'none';  
        e.target.classList.remove('error');  
        e.target.classList.add('valid');  
    }  
});
```

5.5 Financial Data Processing

Processing Steps

javascript

```
class FinancialProcessor {  
    constructor() {  
        this.calculations = new FinancialCalculations();  
        this.riskAssessment = new RiskAssessment();  
        this.recommendations = new RecommendationEngine();  
    }  
  
    async processUserData(userData) {  
        // Step 1: Validate input
```

```
const validationErrors = this.validate(userData);

if (validationErrors.length > 0) {
    throw new Error(validationErrors.join('; '));
}

// Step 2: Perform calculations
const incomeMetrics = this.calculations.calculateIncomeMetrics(userData);
const creditMetrics = this.calculations.calculateCreditMetrics(userData);
const loanMetrics = this.calculations.calculateLoanMetrics(userData);

// Step 3: Assess eligibility
const eligibility = this.assessEligibility(
    incomeMetrics, creditMetrics, loanMetrics
);

// Step 4: Calculate EMI
const emi = this.calculations.calculateEMI(
    userData.loanAmount,
    userData.interestRate,
    userData.tenure
);

// Step 5: Assess risk
const risk = this.riskAssessment.assessRisk(
    userData, incomeMetrics, creditMetrics
);

// Step 6: Generate recommendations
const recommendations = this.recommendations.generate(
    userData, eligibility, emi, risk
);
```



```
    return {  
      eligibility,  
      emi,  
      risk,  
      recommendations,  
      metrics: { incomeMetrics, creditMetrics, loanMetrics }  
    };  
  }  
}
```

Data Structuring

javascript

```
function structureDataForAPI(userData, results) {  
  return {  
    user: {  
      name: userData.name || 'Anonymous',  
      age: userData.age,  
      employmentType: userData.employmentType,  
    },  
    financial: {  
      monthlyIncome: userData.income,  
      creditScore: userData.creditScore,  
      existingEMI: userData.existingEMI || 0,  
      loanAmount: userData.loanAmount,  
      tenure: userData.tenure,  
      interestRate: userData.interestRate || 10,  
    },  
    results: {  
      eligibilityStatus: results.eligibility.status,  
      monthlyEMI: results.emi.monthlyEMI,  
      totalInterest: results.emi.totalInterest,  
    },  
  };  
}
```

```
        totalPayment: results.emi.totalPayment,
        riskLevel: results.risk.level,
        riskScore: results.risk.score,
        recommendations: results.recommendations,
    },
    timestamp: new Date().toISOString()
};
}
```

5.6 Claude AI Integration

AI Integration Process

javascript

```
class AIIntegration {
    constructor(apiKey) {
        this.apiKey = apiKey;
        this.baseUrl = 'https://api.anthropic.com/v1/messages';
    }
}
```

```
async getRecommendations(userData, results) {
    const prompt = this.createPrompt(userData, results);
```

```
    try {
        const response = await fetch(this.baseUrl, {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json',
                'x-api-key': this.apiKey,
                'anthropic-version': '2023-06-01'
            },
            body: JSON.stringify({
                model: 'claude-3-opus-20240229',
```

```

        max_tokens: 1024,
        messages: [{
            role: 'user',
            content: prompt
        }]
    })
});

if (!response.ok) {
    throw new Error(`API Error: ${response.status}`);
}

const data = await response.json();
return this.processRecommendations(data);

} catch (error) {
    console.error('AI Integration Error:', error);
    return this.getFallbackRecommendations(userData);
}
}

```

```

createPrompt(userData, results) {
    return `
    User Financial Profile:
    Monthly Income: ₹${userData.income}
    Employment Type: ${userData.employmentType}
    Age: ${userData.age} years
    Credit Score: ${userData.creditScore}
    Existing EMI: ₹${userData.existingEMI || 0}
    Requested Loan: ₹${userData.loanAmount}
    Loan Tenure: ${userData.tenure} months
    `;
}

```

Calculated EMI: ₹\${results.emi.monthlyEMI}

Risk Level: \${results.risk.level}

Eligibility Status: \${results.eligibility.status}

Please provide:

1. Loan Eligibility Assessment - Explain if the user qualifies and why
2. Financial Risk Analysis - Identify key risk factors
3. Credit Improvement Suggestions - 3 specific actions to improve credit
4. Savings Recommendations - 3 actionable tips for saving
5. Debt Management Strategies - How to manage existing debts
6. Suitable Loan Products - Recommend specific loan types with reasoning

`;

}

}

Recommendation Processing

javascript

```
processRecommendations(aiResponse) {
```

```
    const content = aiResponse.content[0].text;
```

```
    // Parse the AI response into structured recommendations
```

```
    const sections = content.split('\n\n');
```

```
    return {
```

```
        eligibility: sections[0] || 'Eligibility assessment not available',
```

```
        riskAnalysis: sections[1] || 'Risk analysis not available',
```

```
        creditImprovements: this.extractBulletPoints(sections[2]),
```

```
        savingsTips: this.extractBulletPoints(sections[3]),
```

```
        debtStrategies: this.extractBulletPoints(sections[4]),
```

```
        loanProducts: this.extractBulletPoints(sections[5])
```

```
    };
```

```
}
```

5.7 Prompt Engineering Strategies

Prompt Design Principles

Principle	Application
Clarity	Clear, unambiguous instructions
Structure	Organized, logical format
Context	Provide sufficient background information
Specificity	Ask for specific outputs
Actionability	Request actionable recommendations

Prompt Templates

Eligibility Assessment Prompt:

text

User Financial Profile:

[All financial details]

Based on the above profile, please assess loan eligibility by considering:

1. Income stability and adequacy
2. Existing debt obligations
3. Credit history and score
4. Repayment capacity
5. Employment stability

Provide a comprehensive eligibility assessment including:

- Eligibility status (Eligible/Conditional/Not Eligible)
- Reasons for the assessment
- Factors that could improve eligibility
- Maximum loan amount recommendation

Risk Assessment Prompt:

text

User Financial Profile:

[All financial details]

Please analyze financial risk by evaluating:

- 1. Debt-to-income ratio
- 2. Credit history and utilization
- 3. Income stability
- 4. Existing obligations
- 5. Financial behavior patterns

Provide risk assessment including:

- Risk level (Low/Medium/High)
- Key risk factors
- Risk mitigation strategies
- Recommended actions

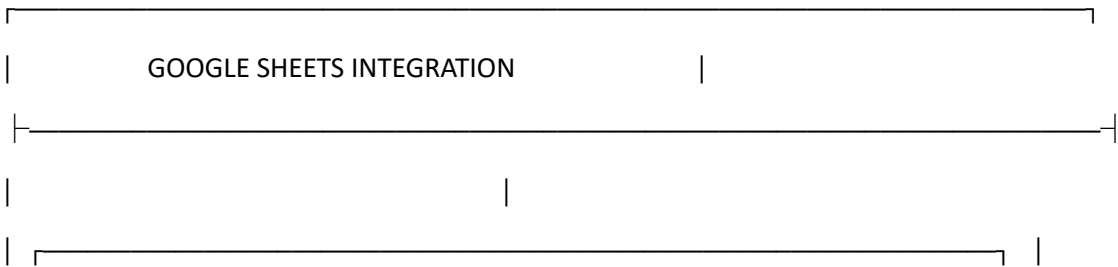
Best Practices

Best Practice	Implementation
Consistency	Use consistent formatting
Completeness	Include all relevant data
Format Specification	Define output format
Examples	Provide examples when helpful
Iteration	Refine prompts based on responses

5.8 Google Sheets Integration

Cloud Storage Architecture

text





Google Apps Script Implementation

javascript

// Google Apps Script - Code.gs

```
function doPost(e) {
```

```
  try {
```

```
    // Parse incoming data
```

```
const data = JSON.parse(e.postData.contents);

// Validate data
const validationResult = validateData(data);
if (!validationResult.valid) {
    return ContentService.createTextOutput(JSON.stringify({
        success: false,
        error: validationResult.error
    })).setMimeType(ContentService.MimeType.JSON);
}
```

```
// Store in spreadsheet
const sheet = SpreadsheetApp.getActiveSheet();
const row = [
    new Date().toISOString(),
    data.user.name || 'Anonymous',
    data.user.age,
    data.financial.monthlyIncome,
    data.financial.employmentType,
    data.financial.creditScore,
    data.financial.existingEMI || 0,
    data.financial.loanAmount,
    data.financial.tenure,
    data.results.eligibilityStatus,
    data.results.monthlyEMI,
    data.results.riskLevel,
    data.results.recommendations.join('; ') || '',
    data.timestamp
];

sheet.appendRow(row);
```



```

    return ContentService.createTextOutput(JSON.stringify({
      success: true,
      message: 'Data stored successfully'
    })).setMimeType(ContentService.MimeType.JSON);

  } catch (error) {
    return ContentService.createTextOutput(JSON.stringify({
      success: false,
      error: error.message
    })).setMimeType(ContentService.MimeType.JSON);
  }
}

function validateData(data) {
  const requiredFields = ['user', 'financial', 'results'];
  for (const field of requiredFields) {
    if (!data[field]) {
      return { valid: false, error: `Missing required field: ${field}` };
    }
  }
  return { valid: true };
}

```

Client-Side Storage Function

javascript

```

async function storeInGoogleSheets(data) {
  const url = 'https://script.google.com/macros/s/your-script-id/exec';

  try {
    const response = await fetch(url, {
      method: 'POST',

```

```

    headers: {
      'Content-Type': 'application/json'
    },
    body: JSON.stringify(data)
  });

  const result = await response.json();

  if (result.success) {
    console.log('Data stored successfully');
    showToast('Data saved to cloud storage', 'success');
  } else {
    console.error('Storage error:', result.error);
    showToast('Unable to save data to cloud', 'warning');
  }

  return result;

} catch (error) {
  console.error('Storage error:', error);
  showToast('Cloud storage connection failed', 'error');
  return { success: false, error: error.message };
}
}

```

5.9 API Communication

Communication Protocol

HTTP Methods:

Method	Use Case	Example
POST	Send data to Claude AI	Financial profile analysis

Method	Use Case	Example
POST	Send data to Google Sheets	Store user records
GET	Retrieve data (future)	Fetch historical data

Request Headers:

javascript

```
const headers = {  
  'Content-Type': 'application/json',  
  'Accept': 'application/json'  
};  
  
// Claude AI headers  
const claudeHeaders = {  
  ...headers,  
  'x-api-key': process.env.CLAUDE_API_KEY,  
  'anthropic-version': '2023-06-01'  
};
```

Error Handling

javascript

```
async function apiRequest(url, options, retryCount = 3) {  
  for (let attempt = 0; attempt < retryCount; attempt++) {  
    try {  
      const response = await fetch(url, options);  
  
      if (!response.ok) {  
        throw new Error(`HTTP Error: ${response.status}`);  
      }  
  
      const data = await response.json();  
      return data;  
    }  
  }  
}
```

```
    } catch (error) {  
        if (attempt === retryCount - 1) {  
            throw error;  
        }  
        // Exponential backoff  
        await new Promise(resolve => setTimeout(resolve, 1000 * Math.pow(2, attempt)));  
    }  
}  
}
```

5.10 Dynamic Dashboard Updates

Update Mechanism

javascript

```
class DashboardManager {  
    constructor() {  
        this.elements = {  
            eligibility: document.getElementById('eligibility-status'),  
            emi: document.getElementById('emi-amount'),  
            risk: document.getElementById('risk-level'),  
            creditScore: document.getElementById('credit-score-display'),  
            recommendations: document.getElementById('ai-recommendations')  
        };  
    }  
  
    updateDashboard(results) {  
        // Update eligibility  
        this.updateEligibility(results.eligibility);  
  
        // Update EMI  
        this.updateEMI(results.emi);  
    }  
}
```

```

// Update risk
this.updateRisk(results.risk);

// Update recommendations
this.updateRecommendations(results.recommendations);

// Animate updates
this.animateUpdates();
}

updateEligibility(eligibility) {
  const element = this.elements.eligibility;
  element.textContent = eligibility.status;
  element.className = `status status-${eligibility.status.toLowerCase()}`;

  // Show detailed explanation
  this.showExplanation(eligibility.reason);
}

updateEMI(emi) {
  const element = this.elements.emi;
  element.textContent = `₹${emi.monthlyEMI.toLocaleString()}`;

  // Update progress bars
  this.updateProgressBars(emi);
}

updateRisk(risk) {
  const element = this.elements.risk;
  element.textContent = risk.level;
  element.className = `risk risk-${risk.level.toLowerCase()}`;
}

```

```
}  
}
```

Real-time Feedback

javascript

```
// Show loading state  
function showLoading() {  
    const dashboard = document.getElementById('dashboard');  
    dashboard.classList.add('loading');  
    document.querySelector('.spinner').style.display = 'block';  
}  
  
// Update results with animation  
function updateResultsWithAnimation(results) {  
    // Hide loading  
    document.querySelector('.spinner').style.display = 'none';  
    dashboard.classList.remove('loading');  
  
    // Fade in results  
    const cards = document.querySelectorAll('.result-card');  
    cards.forEach((card, index) => {  
        setTimeout(() => {  
            card.classList.add('visible');  
            card.style.opacity = '1';  
            card.style.transform = 'translateY(0)';  
        }, 100 * index);  
    });  
}
```

5.11 Security Measures

API Key Protection

javascript

```
// Never embed API keys in frontend code

// Use environment variables

const config = {

  claudeApiKey: process.env.CLAUDE_API_KEY,

  sheetsEndpoint: process.env.SHEETS_ENDPOINT

};
```

```
// Server-side proxy for API calls

async function callClaudeAPI(data) {

  const response = await fetch('/api/claude', {

    method: 'POST',

    body: JSON.stringify(data)

  });

  return response.json();

}
```

Input Sanitization

javascript

```
function sanitizeInput(input) {

  // Remove HTML tags

  let sanitized = input.replace(/<[^>]*>/g, "");

  // Remove script tags

  sanitized = sanitized.replace(/<script\b[^\<]*(?:?!<\script>)<[^\<]*>\script>/gi, "");

  // Escape special characters

  sanitized = sanitized.replace(/[\&<>"]/g, function(match) {

    const escape = {

      '&': '&amp;',

      '<': '&lt;',

      '>': '&gt;',

      '"': '&quot;',


```

```

        ""': '&#x27;'
    };

    return escape[match];

});

return sanitized;
}

```

Security Headers

html

```

<!-- Content Security Policy -->
<meta http-equiv="Content-Security-Policy" content="
    default-src 'self';
    script-src 'self' 'unsafe-inline' https://cdn.jsdelivr.net;
    style-src 'self' 'unsafe-inline' https://fonts.googleapis.com;
    font-src 'self' https://fonts.gstatic.com;
    img-src 'self' data: https://*;
    connect-src 'self' https://api.anthropic.com https://script.google.com;
">

<!-- Referrer Policy -->
<meta name="referrer" content="strict-origin-when-cross-origin">

<!-- Permissions Policy -->
<meta http-equiv="Permissions-Policy" content="
    interest-cohort=(),
    geolocation=(self),
    microphone=(),
    camera=()
">

```

5.12 Error Handling

Comprehensive Error Handling

javascript

```
class ErrorHandler {  
  constructor() {  
    this.errors = [];  
    this.listeners = [];  
  }  
  
  handleError(error, context) {  
    // Log error  
    console.error(`Error in ${context}:`, error);  
  
    // Store error  
    this.errors.push({  
      timestamp: new Date().toISOString(),  
      context,  
      message: error.message,  
      stack: error.stack  
    });  
  
    // Show user-friendly message  
    this.showUserMessage(error, context);  
  
    // Notify listeners  
    this.notifyListeners(error, context);  
  }  
  
  showUserMessage(error, context) {  
    const messages = {  
      network: 'Network connection issue. Please check your internet connection.',  
      validation: 'Please correct the highlighted fields and try again.',
```

```

    api: 'Service temporarily unavailable. Please try again later.',
    default: 'An unexpected error occurred. Please try again.'
  };

  const userMessage = messages[context] || messages.default;
  showToast(userMessage, 'error');
}
}

```

Error Recovery

javascript

```

class ErrorRecovery {
  async tryOperation(operation, fallback = null) {
    try {
      return await operation();
    } catch (error) {
      console.error('Operation failed:', error);

      // Try fallback
      if (fallback) {
        try {
          return await fallback();
        } catch (fallbackError) {
          console.error('Fallback failed:', fallbackError);
          throw fallbackError;
        }
      }

      throw error;
    }
  }
}

```

```

async retryOperation(operation, maxRetries = 3, delay = 1000) {
  for (let attempt = 1; attempt <= maxRetries; attempt++) {
    try {
      return await operation();
    } catch (error) {
      if (attempt === maxRetries) {
        throw error;
      }
      await new Promise(resolve => setTimeout(resolve, delay * attempt));
    }
  }
}

```

5.13 Advantages of Client Logic and Integration

User Benefits

Benefit	Description
Real-time Analysis	Instant financial insights
Intelligent Recommendations	AI-powered personalized advice
Secure Storage	Cloud-based data persistence
Dynamic Interface	Interactive, responsive updates
Improved Responsiveness	Fast, efficient processing

Technical Benefits

Benefit	Description
Scalability	Easy to add new features
Maintainability	Clean, modular code
Performance	Optimized processing
Security	Protected user data

5.14 Future Enhancements

Planned Features

Feature	Description	Priority
Real-time Banking	Direct bank API integration	High
Voice Commands	Voice-based financial queries	Medium
Chat Interface	Natural language financial assistant	Medium
Mobile App	Native iOS and Android apps	High
Multi-language	Support for regional languages	Medium

5.15 Chapter Summary

Key Takeaways

Area	Summary
Architecture	JavaScript-based client logic
Validation	Comprehensive input validation
Processing	Step-by-step financial data processing
AI Integration	Claude AI with prompt engineering
Cloud Storage	Google Sheets via Apps Script
API Communication	Robust API handling
Dashboard Updates	Real-time dynamic updates
Security	API protection and input sanitization
Error Handling	Comprehensive error management

The integration layer successfully connects all components, ensuring seamless communication between the user interface, financial modules, AI services, and cloud storage.